

建構生成式使用者介面 (GenUI) 應用程式

來源：<https://codelabs.developers.google.com/codelabs/genui-intro?hl=zh-tw>

步驟數：9

使用 Flutter、Firebase 和 GenUI 建構工作清單應用程式。

目錄

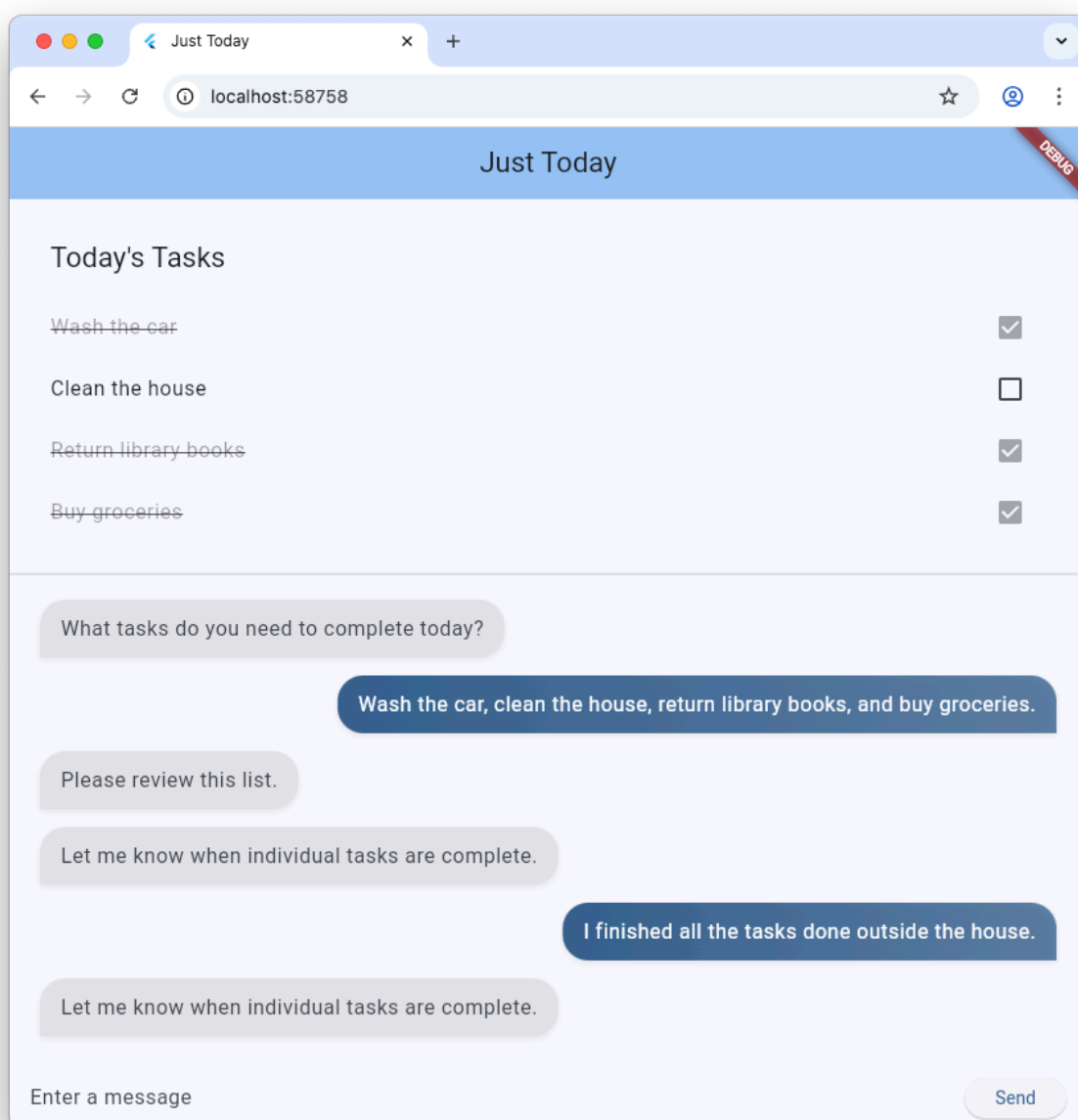
1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 搭建基本對話介面](#)
4. [4. 整合 GenUI 套件](#)
5. [5. 新增等待狀態](#)
6. [6. 保留 GenUI Surface](#)
7. [7. 建構自訂目錄小工具](#)
8. [8. 嘗試以不同方式與代理互動](#)
9. [9. 恭喜](#)

步驟 1 · 約 2 分鐘

1. 簡介

在本程式碼研究室中，您將使用 Flutter、Firebase AI Logic 和新的 `genui` 套件，建構工作清單應用程式。您將從以文字為基礎的聊天應用程式開始，使用 GenUI 升級應用程式，讓代理程式有權建立自己的 UI，最後建構自訂的互動式 UI 元件，您和代理程式可以直接操控。

第一次使用 GenUI 嗎？如果您是第一次聽到「GenUI」這個詞，別擔心！這是「生成式 UI」的簡稱，說明代理程式選擇及建立 UI 元件的使用者體驗。GenUI 可將純文字代理程式，變成可建立下拉式選單、輪轉介面等的代理程式。此外，使用者還能透過這項技術，在習慣使用的 UI 元件之外，以自然語言操作應用程式。詳情請參閱這篇[網誌文章](#)。



學習內容

- 使用 Flutter 和 Firebase AI Logic 建構基本即時通訊介面

- 整合 `genui` 套件，生成 AI 驅動的介面
- 新增進度列，指出應用程式何時等待代理程式回覆
- 建立具名介面，並在 UI 的專屬位置顯示。
- 建立自訂 GenUI 目錄元件，控管工作呈現方式

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 在本機安裝 [Flutter SDK](#)
- 已安裝及設定 [Firebase CLI](#)

本程式碼研究室適用於中階 Flutter 開發人員。

2. 事前準備

設定 Flutter 專案

1. 如果尚未安裝，請在本機安裝 [Flutter SDK](#)。
2. 開啟終端機並執行 `flutter create`，建立新專案：

□□□

```
flutter create intro_to_genui
cd intro_to_genui
```

3. 將必要的依附元件新增至 Flutter 專案：

□□□

```
flutter pub add firebase_core firebase_ai genui json_schema_builder
```

最終的 `dependencies` 區段應如下所示 (版本號碼可能略有不同)：

□□□

```
dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  firebase_core: ^4.9.0
  firebase_ai: ^3.12.1
  genui: ^0.9.0
  json_schema_builder: ^0.1.3
```

4. 執行 `flutter pub get` 下載所有套件。

設定 Firebase 專案

1. 如果尚未安裝，請[安裝 Firebase CLI](#)。
2. 使用 Google 帳戶登入 Firebase：

```
firebase login
```

3. 安裝 FlutterFire CLI：

```
dart pub global activate flutterfire_cli
```

4. 在 Flutter 專案目錄中執行下列指令，設定 Flutter 專案以使用 Firebase：

```
flutterfire configure
```

這項指令會執行下列動作：

1. 詢問您要使用現有 Firebase 專案，還是建立新的 Firebase 專案。選取「建立新的 Firebase 專案」。
2. 詢問您要為 Flutter 應用程式指定哪個平台 (iOS、Android、網頁)。目前請選取「網頁」。

`flutterfire configure` 指令會自動建立 Firebase 專案，並在該專案中建立新的 Firebase 網頁應用程式。接著，這項指令會建立 Firebase 設定檔 (`firebase_options.dart`)，並自動新增至 Flutter 專案的 `lib/` 目錄。

請注意，本程式碼研究室的應用程式只需在電腦上安裝 Flutter SDK 和 Chrome 即可運作 (也就是建構為網頁應用程式)。不過，由於這個應用程式是使用 Flutter 建構，因此也能在其他平台上運作！因此，在程式碼研究室的最後，請嘗試重新執行 `flutterfire configure`，加入對 iOS、Android 或其他平台的支持，然後在該平台上重建應用程式。

詳情請參閱[將 Firebase 新增至 Flutter 應用程式的操作說明](#)。

設定 Firebase AI Logic

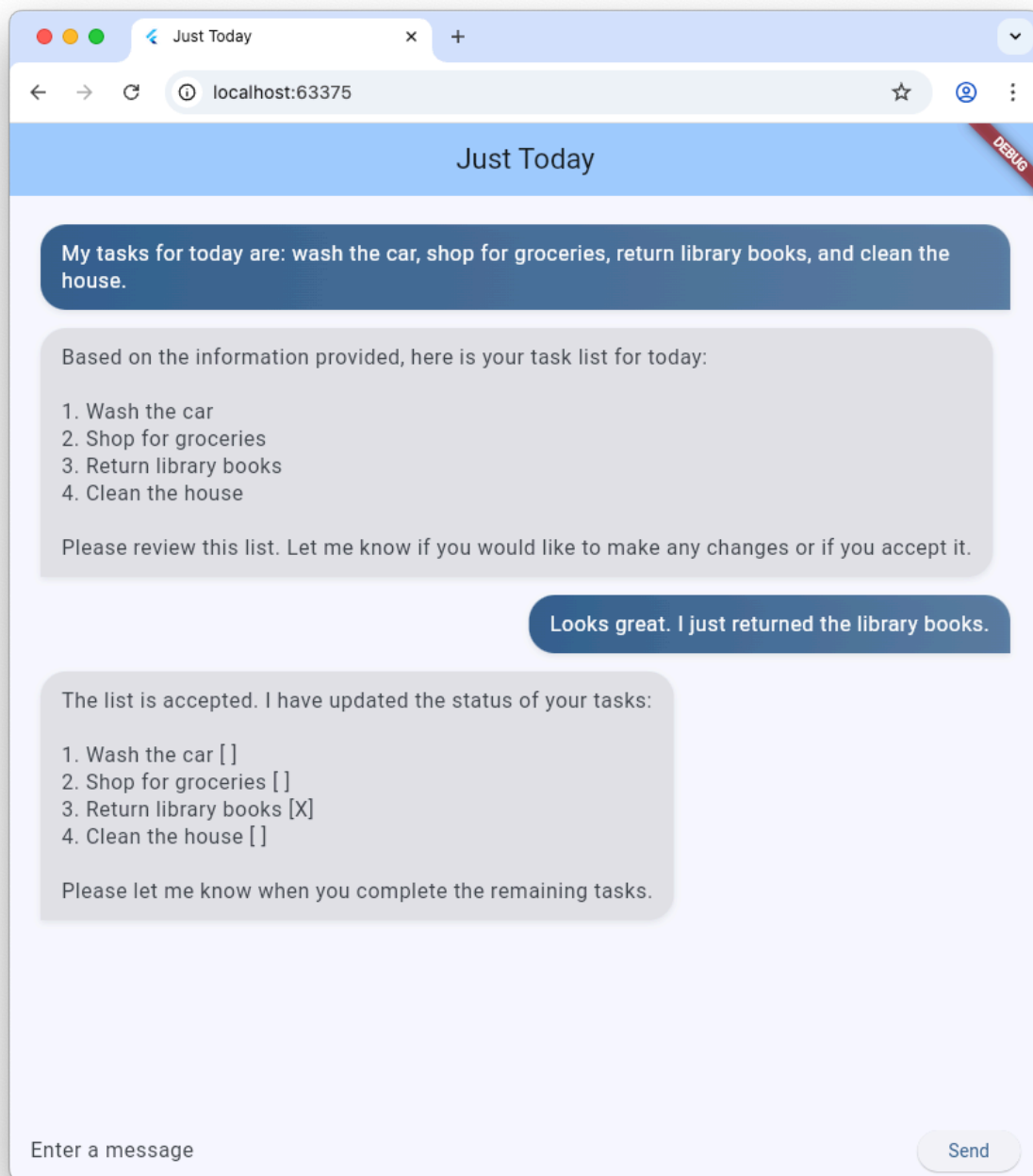
1. 登入 [Firebase 控制台](#)。使用登入 Firebase CLI 的 Google 帳戶。
2. 選取您剛使用 FlutterFire CLI 建立的 Firebase 專案。
3. 在左側導覽選單中，依序選取「AI 服務」>「AI 邏輯」。
4. 按一下「開始使用」，啟動導覽工作流程。
5. 選取要從 **Gemini Developer API** 開始，然後按照畫面上的提示設定 Firebase AI Logic。

您已在「設定 Flutter 專案」一節中取得使用 Firebase AI Logic 的必要 FlutterFire 外掛程式。您已準備好在下一個步驟中開始編寫應用程式程式碼！

步驟 3 · 約 10 分鐘

3. 搭建基本對話介面

在導入生成式 UI 前，您的應用程式需要有基礎：以 Firebase AI Logic 為基礎的文字聊天應用程式。如要快速開始使用，請複製並貼上整個即時通訊介面的設定。



建立訊息泡泡小工具

如要顯示使用者和服務專員的訊息，應用程式需要小工具。建立名為 `lib/message_bubble.dart` 的新檔案，並新增下列類別：

```

import 'package:flutter/material.dart';

class MessageBubble extends StatelessWidget {
  final String text;
  final bool isUser;

  const MessageBubble({super.key, required this.text, required this.isUser});

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final colorScheme = theme.colorScheme;

    final bubbleColor = isUser
      ? colorScheme.primary
      : colorScheme.surfaceContainerHighest;

    final textColor = isUser
      ? colorScheme.onPrimary
      : colorScheme.onSurfaceVariant;

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6.0, horizontal: 8.0),
      child: Column(
        crossAxisAlignment: isUser
          ? CrossAxisAlignment.end
          : CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: isUser
              ? MainAxisAlignment.end
              : MainAxisAlignment.start,
            children: [
              Flexible(
                child: Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 16.0,
                    vertical: 12.0,
                  ),
                ),
                decoration: BoxDecoration(
                  color: bubbleColor,
                  borderRadius: BorderRadius.only(
                    topLeft: const Radius.circular(20),
                    topRight: const Radius.circular(20),
                    bottomLeft: Radius.circular(isUser ? 20 : 0),
                    bottomRight: Radius.circular(isUser ? 0 : 20),
                  ),
                ),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withAlpha(20),
                    blurRadius: 4,
                    offset: const Offset(0, 2),

```



```

        ),
      ],
      gradient: isUser
        ? LinearGradient(
            colors: [
              colorScheme.primary,
              colorScheme.primary.withAlpha(200),
            ],
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
          )
        : null,
    ),
    child: Text(
      text,
      style: theme.textTheme.bodyLarge?.copyWith(
        color: textColor,
        height: 1.3,
      ),
    ),
  ),
),
],
),
const SizedBox(height: 2),
],
),
);
}
}

```

`MessageBubble` 是顯示單一即時通訊訊息的 `StatelessWidget`。本程式碼研究室稍後會使用這個小工具顯示你和代理程式的訊息，但它基本上只是花俏的 `Text` 小工具。

在 `main.dart` 中實作 Chat UI

將 `lib/main.dart` 的所有內容替換為以下完整的文字聊天機器人實作內容：

```

import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_ai/firebase_ai.dart';
import 'package:intro_to_genui/message_bubble.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Just Today',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
      ),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

sealed class ConversationItem {}

class TextItem extends ConversationItem {
  final String text;
  final bool isUser;
  TextItem({required this.text, this.isUser = false});
}

class _MyHomePageState extends State<MyHomePage> {
  final List<ConversationItem> _items = [];
  final _textController = TextEditingController();
  final _scrollController = ScrollController();
  late final ChatSession _chatSession;

  @override
  void initState() {
    super.initState();
    final model = FirebaseAI.googleAI().generativeModel(

```

```

        model: 'gemini-3-flash-preview',
    );
    _chatSession = model.startChat();
    _chatSession.sendMessage(Content.text(systemInstruction));
}

void _scrollToBottom() {
    WidgetsBinding.instance.addPostFrameCallback((_) {
        if (_scrollController.hasClients) {
            _scrollController.animateTo(
                _scrollController.position.maxScrollExtent,
                duration: const Duration(milliseconds: 300),
                curve: Curves.easeOut,
            );
        }
    });
}

@override
void dispose() {
    _textController.dispose();
    _scrollController.dispose();
    super.dispose();
}

Future<void> _addMessage() async {
    final text = _textController.text;

    if (text.trim().isEmpty) {
        return;
    }
    _textController.clear();

    setState(() {
        _items.add(TextItem(text: text, isUser: true));
    });

    _scrollToBottom();

    final response = await _chatSession.sendMessage(Content.text(text));

    if (response.text?.isNotEmpty ?? false) {
        setState(() {
            _items.add(TextItem(text: response.text!, isUser: false));
        });
        _scrollToBottom();
    }
}

@override
Widget build(BuildContext context) {
    return Scaffold(

```

```

appBar: AppBar(
  backgroundColor: Theme.of(context).colorScheme.inversePrimary,
  title: const Text('Just Today'),
),
body: Column(
  children: [
    Expanded(
      child: ListView(
        controller: _scrollController,
        padding: const EdgeInsets.all(16),
        children: [
          for (final item in _items)
            switch (item) {
              TextItem() => MessageBubble(
                text: item.text,
                isUser: item.isUser,
              ),
            },
        ],
      ),
    ),
    SafeArea(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16.0),
        child: Row(
          children: [
            Expanded(
              child: TextField(
                controller: _textController,
                onSubmitted: (_) => _addMessage(),
                decoration: const InputDecoration(
                  hintText: 'Enter a message',
                ),
              ),
            ),
            const SizedBox(width: 8),
            ElevatedButton(
              onPressed: _addMessage,
              child: const Text('Send'),
            ),
          ],
        ),
      ),
    ),
  ],
),
);
}
}

```

```

const systemInstruction = '''
  ### PERSONA

```

```
You are an expert task planner.
```

```
## GOAL
```

```
Work with me to produce a list of tasks that I should do today, and then track the completion status of each one.
```

```
## RULES
```

```
Talk with me only about tasks that I should do today.
```

```
Do not engage in conversation about any other topic.
```

```
Do not offer suggestions unless I ask for them.
```

```
Do not offer encouragement unless I ask for it.
```

```
Do not offer advice unless I ask for it.
```

```
Do not offer opinions unless I ask for them.
```

```
## PROCESS
```

```
### Planning
```

```
* Ask me for information about tasks that I should do today.
```

```
* Synthesize a list of tasks from that information.
```

```
* Ask clarifying questions if you need to.
```

```
* When you have a list of tasks that you think I should do today, present it to me for review.
```

```
* Respond to my suggestions for changes, if I have any, until I accept the list.
```

```
### Tracking
```

```
* Once the list is accepted, ask me to let you know when individual tasks are complete.
```

```
* If I tell you a task is complete, mark it as complete.
```

```
* Once all tasks are complete, send a message acknowledging that, and then end the conversation.
```

```
''';
```

您剛才複製並貼上的 `main.dart` 檔案會使用 Firebase AI Logic 和 `systemInstruction` 中的提示，設定基本的 `ChatSession`。這項功能會維護 `TextItem` 元素清單，並使用您先前建立的 `MessageBubble` 小工具，與使用者查詢一併顯示，藉此管理對話輪次。

繼續操作前，請先檢查下列事項：

- `initState` 方法用於設定與 Firebase AI Logic 的連線。
- 應用程式會提供 `TextField` 和按鈕，方便您傳送訊息給服務專員。
- `_addMessage` 方法會將使用者的訊息傳送給服務專員。
- 「`_items`」清單會儲存對話記錄。
- 訊息會使用 `MessageBubble` 小工具顯示在 `ListView` 中。

測試應用程式

完成上述步驟後，您就可以執行並測試應用程式。

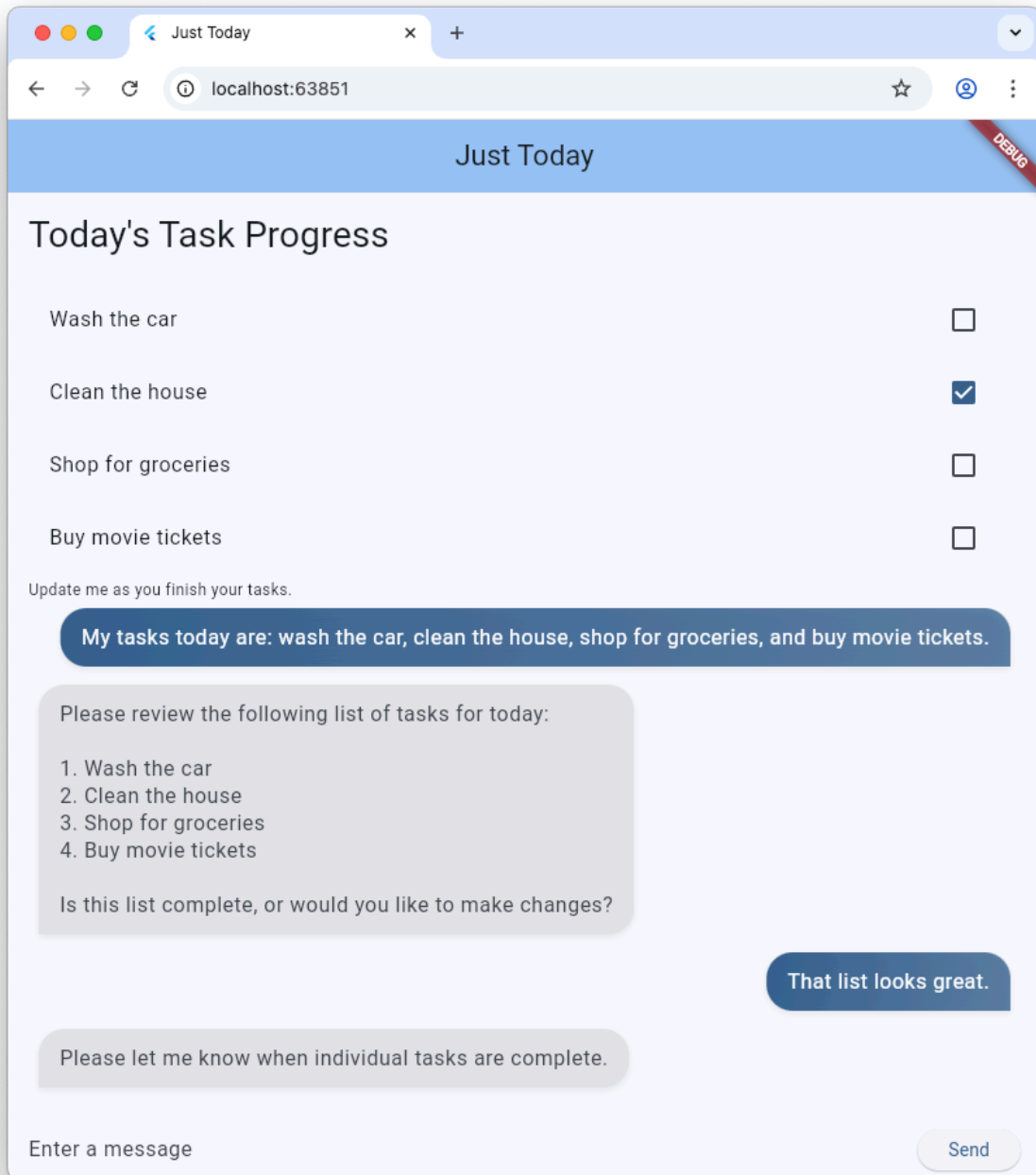
```
flutter run -d chrome
```

試著與代理討論今天想完成的任務。純文字介面雖然也能完成工作，但 GenUI 可讓體驗更輕鬆快速。

步驟 4 · 約 15 分鐘

4. 整合 GenUI 套件

現在請從純文字升級至生成式 UI。您會將基本的 Firebase 訊息傳送迴圈，換成 GenUI `Conversation`、`Catalog` 和 `SurfaceController` 物件。這項功能可讓 AI 模型在即時通訊串流中例項化實際的 Flutter 小工具。



`genui` 套件提供五個類別，您會在整個程式碼研究室中使用這些類別：

- `SurfaceController` 會將模型產生的 UI 對應至畫面。
- `A2uiTransportAdapter` 可將內部 GenUI 要求與任何外部語言模型連結。
- `Conversation` 會為 Flutter 應用程式的控制器和傳輸轉接程式，包裝單一整合式 API。
- `Catalog` 說明語言模型可用的小工具和屬性。
- `Surface` 是用來顯示模型生成 UI 的小工具。

準備好顯示生成的 `Surface`

現有程式碼包含 `TextItem` 類別，代表對話中的單一訊息。新增另一個類別，代表代理程式建立的 `Surface`：

```
class SurfaceItem extends ConversationItem {
  final String surfaceId;
  SurfaceItem({required this.surfaceId});
}
```

初始化 GenUI 建構模塊

在 `lib/main.dart` 頂端匯入 `genui` 程式庫：

```
import 'package:genui/genui.dart' hide TextPart;
import 'package:genui/genui.dart' as genui;
```

`genui` 和 `firebase_ai` 套件都包含 `TextPart` 類別。以這種方式匯入 `genui` 時，您會將 `TextPart` 的版本命名空間設為 `genui.TextPart`，避免名稱衝突。

在 `_chatSession` 後方的 `_MyHomePageState` 中宣告核心功能控制器：

```
class _MyHomePageState extends State<MyHomePage> {
  // ... existing members
  late final ChatSession _chatSession;

  // Add GenUI controllers
  late final SurfaceController _controller;
  late final A2uiTransportAdapter _transport;
  late final Conversation _conversation;
  late final Catalog catalog;
```

接著，更新 `initState` 以準備 GenUI 程式庫的控制器。

從 `initState` 移除這一行：

```
_chatSession.sendMessage(Content.text(systemInstruction));
```

然後新增下列程式碼：


```

@Override
void initState() {
    // ... existing code ...

    // Initialize the GenUI Catalog.
    // The genui package provides a default set of primitive widgets (like text
    // and basic buttons) out of the box using this class.
    catalog = BasicCatalogItems.asCatalog();

    // Create a SurfaceController to manage the state of generated surfaces.
    _controller = SurfaceController(catalogs: [catalog]);

    // Create a transport adapter that will process messages to and from the
    // agent, looking for A2UI messages.
    _transport = A2uiTransportAdapter(onSend: _sendAndReceive);

    // Link the transport and SurfaceController together in a Conversation,
    // which provides your app a unified API for interacting with the agent.
    _conversation = Conversation(
        controller: _controller,
        transport: _transport,
    );
}

```

這段程式碼會建立 `Conversation` facade，用於管理控制器和轉接器。該對話會提供一連串事件，讓應用程式掌握代理程式的建立內容，以及傳送訊息給代理程式的方法。

接著，建立對話事件的接聽程式。包括與介面相關的事件，以及簡訊和錯誤的事件：

```

@override
void initState() {
  // ... existing code ...

  // Listen to GenUI stream events to update the UI
  _conversation.events.listen((event) {
    setState(() {
      switch (event) {
        case ConversationSurfaceAdded added:
          _items.add(SurfaceItem(surfaceId: added.surfaceId));
          _scrollToBottom();
        case ConversationSurfaceRemoved removed:
          _items.removeWhere(
            (item) =>
              item is SurfaceItem && item.surfaceId == removed.surfaceId,
          );
        case ConversationContentReceived content:
          _items.add(TextItem(text: content.text, isUser: false));
          _scrollToBottom();
        case ConversationError error:
          debugPrint('GenUI Error: ${error.error}');
        default:
      }
    });
  });
}

```

最後，建立系統提示並傳送給代理程式：

```

@override
void initState() {
  // ... existing code ...

  // Create the system prompt for the agent, which will include this app's
  // system instruction as well as the schema for the catalog.
  final promptBuilder = PromptBuilder.chat(
    catalog: catalog,
    systemPromptFragments: [systemInstruction],
  );

  // Send the prompt into the Conversation, which will subsequently route it
  // to Firebase using the transport mechanism.
  _conversation.sendRequest(
    ChatMessage.system(promptBuilder.systemPromptJoined()),
  );
}

```

顯示介面

接著，更新 `ListView` 的 `build` 方法，在 `_items` 清單中顯示 `SurfaceItem`：

```
Expanded(
  child: ListView(
    controller: _scrollController,
    padding: const EdgeInsets.all(16),
    children: [
      for (final item in _items)
        switch (item) {
          TextItem() => MessageBubble(
            text: item.text,
            isUser: item.isUser,
          ),
          // New!
          SurfaceItem() => Surface(
            surfaceContext: _controller.contextFor(
              item.surfaceId,
            ),
          ),
        },
    ],
  ),
),
```

`Surface` 小工具的建構函式會採用 `surfaceContext`，告知小工具負責顯示哪個介面。先前建立的 `SurfaceController` (即 `_controller`) 會提供每個介面的定義和狀態，並確保更新時會重建。

將 GenUI 連結至 Firebase AI Logic

`genui` 套件採用「自備模型」方法，也就是由您控管為體驗提供支援的 LLM。在本例中，您使用的是 Firebase AI Logic，但這個套件可與各種代理程式和供應商搭配使用。

這項自由度會帶來一些額外責任：您需要擷取 `genui` 套件產生的訊息，並傳送至所選代理程式；您也需要擷取代理程式的回應，並傳送回 `genui`。

為此，您要定義上一步程式碼中參照的 `_sendAndReceive` 方法。在 `MyHomePageState` 中加入這段程式碼：

```

Future<void> _sendAndReceive(ChatMessage msg) async {
  final buffer = StringBuffer();

  // Reconstruct the message part fragments
  for (final part in msg.parts) {
    if (part.isUiInteractionPart) {
      buffer.write(part.asUiInteractionPart!.interaction);
    } else if (part is genui.TextPart) {
      buffer.write(part.text);
    }
  }

  if (buffer.isEmpty) {
    return;
  }

  final text = buffer.toString();

  // Send the string to Firebase AI Logic.
  final response = await _chatSession.sendMessage(Content.text(text));

  if (response.text?.isNotEmpty ?? false) {
    // Feed the response back into GenUI's transportation layer
    _transport.addChunk(response.text!);
  }
}

```

每當需要傳送訊息給代理程式時，`genui` 套件就會呼叫這個方法。方法結尾的 `addChunk` 呼叫會將代理的回應傳回 `genui` 套件，讓套件處理回應並產生 UI。

最後，請以這個新版本完全取代現有的 `_addMessage` 方法，這樣訊息就會改為傳送至 `Conversation`，而不是直接傳送至 Firebase：

```

Future<void> _addMessage() async {
  final text = _textController.text;

  if (text.trim().isEmpty) {
    return;
  }

  _textController.clear();

  setState(() {
    _items.add(TextItem(text: text, isUser: true));
  });

  _scrollToBottom();

  // Send the user's input through GenUI instead of directly to Firebase.
  await _conversation.sendRequest(ChatMessage.user(text));
}

```

大功告成！再次嘗試執行應用程式。除了文字訊息，您還會看到代理程式生成按鈕、文字小工具等 UI 介面。

你甚至可以要求代理程式以特定方式顯示 UI。舉例來說，你可以輸入「請以直欄顯示我的工作，並提供按鈕將每項工作標示為完成」。

5. 新增等待狀態

LLM 生成作業為非同步。等待回覆時，即時通訊介面需要停用輸入按鈕並顯示進度指標，讓使用者知道 GenUI 正在建立內容。幸好，`genui` 套件提供 `Listenable`，可用於追蹤對話狀態。該 `ConversationState` 值包含 `isWaiting` 屬性，可判斷模型是否正在生成內容。

使用 `ValueListenableBuilder` 包裝輸入控制項

在 `lib/main.dart` 的底部建立 `ValueListenableBuilder`，包裝 `Row` (其中包含 `TextField` 和 `ElevatedButton`)，以便監聽 `_conversation.state`。檢查 `state.isWaiting` 即可在模型生成內容時停用輸入功能。

```
ValueListenableBuilder<ConversationState>(
  valueListenable: _conversation.state,
  builder: (context, state, child) {
    return Row(
      children: [
        Expanded(
          child: TextField(
            controller: _textController,
            // Also disable the Enter key submission when waiting!
            onSubmitted: state.isWaiting ? null : (_) => _addMessage(),
            decoration: const InputDecoration(
              hintText: 'Enter a message',
            ),
          ),
        ),
        const SizedBox(width: 8),
        ElevatedButton(
          // Disable the send button when the model is generating
          onPressed: state.isWaiting ? null : _addMessage,
          child: const Text('Send'),
        ),
      ],
    );
  },
),
```

新增進度列

將主要 `Column` 小工具包裝在 `Stack` 內，並將 `LinearProgressIndicator` 新增為該堆疊的第二個子項，錨定在底部。完成後，`Scaffold` 的 `body` 應如下所示：

```

body: Stack( // New!
  children: [
    Column(
      children: [
        Expanded(
          child: ListView(
            controller: _scrollController,
            padding: const EdgeInsets.all(16),
            children: [
              for (final item in _items)
                switch (item) {
                  TextItem() => MessageBubble(
                    text: item.text,
                    isUser: item.isUser,
                  ),
                  SurfaceItem() => Surface(
                    surfaceContext: _controller.contextFor(
                      item.surfaceId,
                    ),
                  ),
                },
            ],
          ),
        ),
      ],
    SafeArea(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16.0),
        child: ValueListenableBuilder<ConversationState>(
          valueListenable: _conversation.state,
          builder: (context, state, child) {
            return Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _textController,
                    onSubmitted:
                      state.isWaiting ? null : (_) => _addMessage(),
                    decoration: const InputDecoration(
                      hintText: 'Enter a message',
                    ),
                  ),
                const SizedBox(width: 8),
                ElevatedButton(
                  onPressed: state.isWaiting ? null : _addMessage,
                  child: const Text('Send'),
                ),
              ],
            );
          },
        ),
      ),
    ),
  ],
)

```

```
    ),  
  ],  
),  
// Listen to the state again, this time to render a progress indicator  
ValueListenableBuilder<ConversationState>(  
  valueListenable: _conversation.state,  
  builder: (context, state, child) {  
    if (state.isWaiting) {  
      return const LinearProgressIndicator();  
    }  
    return const SizedBox.shrink();  
  },  
),  
],  
),  
),
```

6. 保留 GenUI Surface

目前，工作清單會顯示在捲動的即時通訊訊息串中，每當有新訊息或介面出現，就會附加到清單中。在下一個步驟中，您將瞭解如何命名介面，並在 UI 中的特定位置顯示介面。

首先，在 `main.dart` 頂端，於 `void main()` 之前，宣告要用做介面 ID 的常數：

```
const taskDisplaySurfaceId = 'task_display';
```

接著，更新 `Conversation` 監聽器中的 `switch` 陳述式，確保具有該 ID 的任何介面都不會新增至 `_items`：

```
case ConversationSurfaceAdded added:
  if (added.surfaceId != taskDisplaySurfaceId) {
    _items.add(SurfaceItem(surfaceId: added.surfaceId));
    _scrollToBottom();
  }
```

接著，開啟小工具樹狀結構的版面配置結構，在聊天記錄正上方建立固定介面的空間。將這兩個小工具新增為主要 `Column` 的第一個子項：

```
AnimatedSize(
  duration: const Duration(milliseconds: 300),
  child: Container(
    padding: const EdgeInsets.all(16),
    alignment: Alignment.topLeft,
    child: Surface(
      surfaceContext: _controller.contextFor(
        taskDisplaySurfaceId,
      ),
    ),
  ),
),
const Divider(),
```

到目前為止，代理可自由建立及使用介面。如要提供更具體的指示，請返回系統提示。在 `systemInstruction` 常數中儲存的提示結尾，新增下列 `## USER INTERFACE` 區段：

```
const systemInstruction = '''
  // ... existing prompt content ...

  ## USER INTERFACE
  *   To display the list of tasks create one and only one instance of the
      TaskDisplay catalog item. Use "$taskDisplaySurfaceId" as its surface ID.
  *   Update $taskDisplaySurfaceId as necessary when the list changes.
  *   $taskDisplaySurfaceId must include a button for each task that I can use
      to mark it complete. When I use that button to mark a task complete, it
      should send you a message indicating what I've done.
  *   Avoid repeating the same information in a single message.
  *   When responding with text, rather than A2UI messages, be brief.
  ''';
```

請務必為代理提供明確的指令，說明何時及如何使用 UI 介面。只要告知代理程式使用特定目錄項目和介面 ID (並重複使用單一例項)，即可確保代理程式建立您想看到的介面。

您還有更多工作要完成，但可以嘗試再次執行應用程式，看看代理是否會在 UI 頂端建立工作顯示介面。

7. 建構自訂目錄小工具

此時 `TaskDisplay` 目錄項目不存在。在接下來的幾個步驟中，您將建立資料結構定義、剖析該結構定義的類別、小工具，以及將所有內容整合在一起的目錄項目，藉此修正這個問題。

首先，建立名為 `task_display.dart` 的檔案，並新增下列匯入項目：

```
import 'package:flutter/material.dart';
import 'package:genui/genui.dart';
import 'package:json_schema_builder/json_schema_builder.dart';
```

建立資料結構定義

接著，定義代理程式在建立工作畫面時提供的資料結構定義。這個程序會使用 `json_schema_builder` 套件中的一些精巧建構函式，但基本上您只是在定義訊息中使用的 JSON 結構定義，這些訊息會傳送給代理程式或從代理程式傳送。

請先從參照元件名稱的基本 `S.object` 開始：

```
final taskDisplaySchema = S.object(
  properties: {
    'component': S.string(enumValues: ['TaskDisplay']),
  },
);
```

接著，將 `title`、`tasks`、`name`、`isCompleted` 和 `completeAction` 新增至結構定義屬性。

```
final taskDisplaySchema = S.object(
  properties: {
    'component': S.string(enumValues: ['TaskDisplay']),
    'title': S.string(description: 'The title of the task list'),
    'tasks': S.list(
      description: 'A list of tasks to be completed today',
      items: S.object(
        properties: {
          'name': S.string(description: 'The name of the task to be completed'),
          'isCompleted': S.boolean(
            description: 'Whether the task is completed',
          ),
          'completeAction': A2uiSchemas.action(
            description:
              'The action performed when the user has completed the task.',
          ),
        },
      ),
    },
  ),
);
```

請查看 `completeAction` 屬性。這是使用 `A2uiSchemas.action` 建立的，也就是代表 A2UI 動作的結構定義屬性建構函式。在結構定義中加入動作後，應用程式基本上會告訴代理程式：「嘿，當你給我工作時，請一併提供動作的名稱和中繼資料，我可以用這些資料告訴你工作已完成。」稍後，當使用者輕觸核取方塊時，應用程式就會叫用該動作。

接著，將 `required` 欄位新增至結構定義。這些屬性會指示代理程式每次都填入特定屬性。在這種情況下，所有屬性都是必填！

```
final taskDisplaySchema = S.object(  
  properties: {  
    'component': S.string(enumValues: ['TaskDisplay']),  
    'title': S.string(description: 'The title of the task list'),  
    'tasks': S.list(  
      description: 'A list of tasks to be completed today',  
      items: S.object(  
        properties: {  
          'name': S.string(description: 'The name of the task to be completed'),  
          'isCompleted': S.boolean(  
            description: 'Whether the task is completed',  
          ),  
          'completeAction': A2uiSchemas.action(  
            description:  
              'The action performed when the user has completed the task.',  
          ),  
        },  
        // New!  
        required: ['name', 'isCompleted', 'completeAction'],  
      ),  
    ),  
  },  
  // New!  
  required: ['title', 'tasks'],  
);
```

建立資料剖析類別

建立這個元件的例項時，代理程式會傳送符合架構的資料。新增兩個類別，將傳入的 JSON 剖析為強型別的 Dart 物件。請注意 `_TaskDisplayData` 如何處理根結構，同時將內部陣列剖析作業委派給 `_TaskData`。

```

class _TaskData {
    final String name;
    final bool isCompleted;
    final String actionName;
    final JsonMap actionContext;

    _TaskData({
        required this.name,
        required this.isCompleted,
        required this.actionName,
        required this.actionContext,
    });

    factory _TaskData.fromJson(Map<String, Object?> json) {
        try {
            final action = json['completeAction']! as JsonMap;
            final event = action['event']! as JsonMap;

            return _TaskData(
                name: json['name'] as String,
                isCompleted: json['isCompleted'] as bool,
                actionName: event['name'] as String,
                actionContext: event['context'] as JsonMap,
            );
        } catch (e) {
            throw Exception('Invalid JSON for _TaskData: $e');
        }
    }
}

class _TaskDisplayData {
    final String title;
    final List<_TaskData> tasks;

    _TaskDisplayData({required this.title, required this.tasks});

    factory _TaskDisplayData.fromJson(Map<String, Object?> json) {
        try {
            return _TaskDisplayData(
                title: (json['title'] as String?) ?? 'Tasks',
                tasks: (json['tasks'] as List<Object?>)
                    .map((e) => _TaskData.fromJson(e as Map<String, Object?>))
                    .toList(),
            );
        } catch (e) {
            throw Exception('Invalid JSON for _TaskDisplayData: $e');
        }
    }
}

```

如果您曾使用 Flutter 建構應用程式，這些類別可能與您建立的類別相似。這些函式會接受 `JsonMap`，並傳回包含從 JSON 剖析資料的強型別物件。

查看 `_TaskData` 中的 `actionName` 和 `actionContext` 欄位。這些屬性是從 JSON 的 `completeAction` 屬性中擷取，包含動作名稱和資料環境 (參照 GenUI 資料模型中的動作位置)。稍後會用來建立 `UserActionEvent`。

資料模型是集中式可觀察的儲存空間，用於存放所有動態 UI 狀態，由 `genui` 程式庫維護。當代理程式從目錄建立 UI 元件時，也會建立符合元件結構定義的資料物件。這個資料物件會儲存在用戶端的資料模型中，因此可用於建構小工具，並在後續傳送給代理程式的訊息中參照 (例如您即將連結至小工具的 `completeAction`)。

新增小工具

現在，請建立小工具來顯示清單。這個函式應接受 `_TaskDisplayData` 類別的例項，以及工作完成時要叫用的回呼。

```

class _TaskDisplay extends StatelessWidget {
  final _TaskDisplayData data;
  final void Function(_TaskData) onCompleteTask;

  const _TaskDisplay({required this.data, required this.onCompleteTask});

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Padding(
          padding: const EdgeInsets.all(16.0),
          child: Text(
            data.title,
            style: Theme.of(context).textTheme.titleLarge,
          ),
        ),
        ...data.tasks.map(
          (task) => CheckboxListTile(
            title: Text(
              task.name,
              style: TextStyle(
                decoration: task.isCompleted
                  ? TextDecoration.lineThrough
                  : TextDecoration.none,
              ),
            ),
            value: task.isCompleted,
            onChanged: task.isCompleted
              ? null
              : (val) {
                  if (val == true) {
                    onCompleteTask(task);
                  }
                },
          ),
        ),
      ],
    );
  }
}

```

建立 CatalogItem

建立結構定義、剖析器和小工具後，現在可以建立 `CatalogItem`，將這些項目全部繫結在一起。

在 `task_display.dart` 底部，將 `taskDisplay` 建立為頂層變數，使用 `_TaskDisplayData` 剖析傳入的 JSON，並建構 `_TaskDisplay` 小工具的例項。

```
final taskDisplay = CatalogItem(
  name: 'TaskDisplay',
  dataSchema: taskDisplaySchema,
  widgetBuilder: (itemContext) {
    final json = itemContext.data as Map<String, Object?>;
    final data = _TaskDisplayData.fromJson(json);

    return _TaskDisplay(
      data: data,
      onCompleteTask: (task) async {
        // We will implement this next!
      },
    );
  },
);
```

實作 onCompleteTask

如要讓小工具正常運作，必須在工作完成時回報給代理程式。使用下列程式碼取代空白的 `onCompleteTask` 預留位置，透過工作資料中的 `completeAction` 建立及傳送事件。

```
onCompleteTask: (task) async {
  // A data context is a reference to a location in the data model. This line
  // turns that reference into a concrete data object that the agent can use.
  // It's kind of like taking a pointer and replacing it with the value it
  // points to.
  final JsonMap resolvedContext = await resolveContext(
    itemContext.dataContext,
    task.actionContext,
  );

  // Dispatch an event back to the agent, letting it know a task was completed.
  // This will be sent to the agent in an A2UI message that includes the name
  // of the action, the surface ID, and the resolved data context.
  itemContext.dispatchEvent(
    UserActionEvent(
      name: task.actionName,
      sourceComponentId: itemContext.id,
      context: resolvedContext,
    ),
  );
}
```

註冊目錄項目

最後，開啟 `main.dart`，匯入新檔案，並與其他目錄項目一起註冊。

在 `lib/main.dart` 頂端新增這項匯入項目：

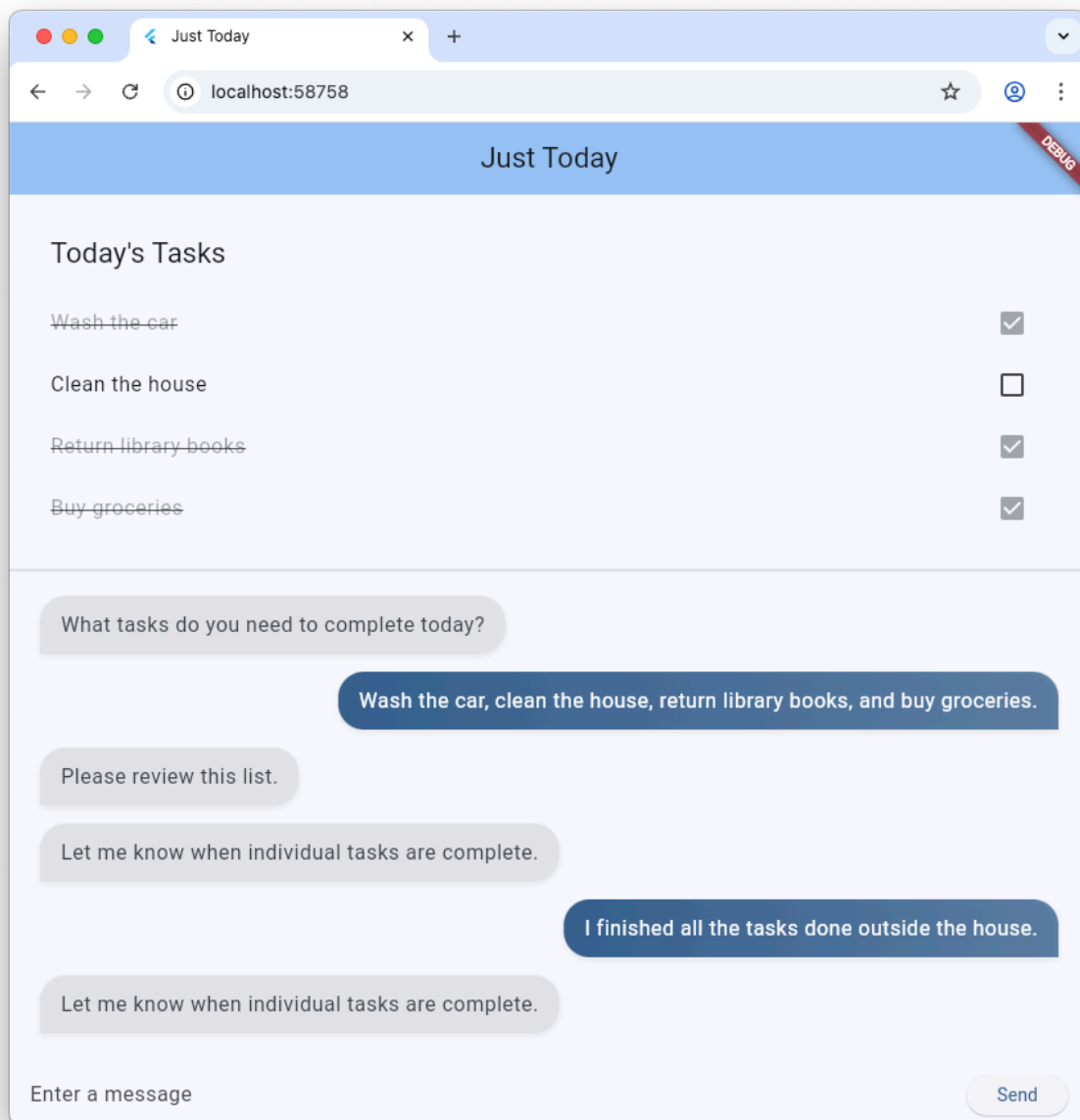
```
import 'task_display.dart';
```


將 `initState()` 函式中的 `catalog = BasicCatalogItems.asCatalog();` 替換為：

```
// The Catalog is immutable, so use copyWith to create a new version
// that includes our custom catalog item along with the basics.
catalog = BasicCatalogItems.asCatalog().copyWith(newItems: [taskDisplay]);
```

大功告成！熱重新啟動應用程式，即可查看變更。

8. 嘗試以不同方式與代理互動



現在您已將新小工具新增至目錄，並在應用程式的 UI 中為小工具預留空間，接下來就能開始使用代理程式。GenUI 的主要優點之一，是提供兩種與資料互動的方式：透過應用程式 UI (例如按鈕和核取方塊)，以及透過可理解自然語言並推論資料的代理程式。建議您兩種都試試看！

- 在文字欄位中描述三到四項工作，然後查看清單中顯示的內容。
- 使用核取方塊將工作切換為完成或未完成。
- 列出 5 到 6 項工作，然後要求代理程式移除需要開車前往某處的工作。
- 請智慧助理將重複性工作列為個別項目 (例如「我要為媽媽、爸爸和奶奶買節慶卡片。請為這些項目分別建立工作。」)。
- 請代理人將所有工作標示為完成或未完成，或勾選前兩到三項工作。

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已使用生成式 UI 和 Flutter 建構 AI 輔助的工作追蹤應用程式。

目前所學內容

- 使用 Flutter Firebase SDK 與 Google 基礎模型互動
- 使用 GenUI 算繪 Gemini 生成的互動式介面
- 使用預先決定的靜態算繪 ID，將版面配置中的介面固定
- 設計自訂結構定義和微件目錄，打造穩健的互動迴圈

參考文件

- [GitHub 上的 GenUI 存放區](#)
 - [A2UI 通訊協定](#)
-